

Design: Provider extensions as installable Android APKs

Revision: 1 **Last modified:** 2026-06-08T00:00:00Z **Status:** design proposal (no build wiring yet — future direction) **Classification:** project-specific

Operator goal: *"add new providers by installing Android extension APKs ... providers MAY be hosted/published to Google Play Store ... by the same or community-driven teams."*

1. Where we are today (grounded in real code)

Lava's tracker SDK has a **clean feature-interface abstraction** but a **compile-time-coupled registry**:

- `core/tracker/api/TrackerDescriptor.kt` — `trackerId`, `displayName`, `baseUrls`, `capabilities: Set<TrackerCapability>`, `authType`, `fail-closed flags` (`verified`, `supportsAnonymous`, `apiSupported`).
- `core/tracker/api/TrackerClient.kt:16` —
`fun <T : TrackerFeature> getFeature(featureClass: KClass<T>): T?` — the capability gate: returns non-null only if the matching `TrackerCapability` is declared (§6.E honesty).
- `core/tracker/api/TrackerCapability.kt` — 13-value enum (`SEARCH`, `BROWSE`, `FORUM`, `TOPIC`, `COMMENTS`, `FAVORITES`, `TORRENT_DOWNLOAD`, `MAGNET_LINK`, `AUTH_REQUIRED`, `CAPTCHA_LOGIN`, `RSS`, `UPLOAD`, `USER_PROFILE`).
- `core/tracker/api/feature/*Tracker.kt` — 7 feature interfaces (`Searchable` / `Browsable` / `Topic` / `Comments` / `Favorites` / `Authenticatable` / `Downloadable`).
- **The coupling point:** `core/tracker/client/di/TrackerClientModule.kt:260–276` `provideTrackerRegistry(...)` @Injects the 6 concrete factories by name and calls `register()` on each. **No service-loader, no reflection, no runtime discovery** — adding a provider today means editing this method + `settings.gradle.kts`.

So the *contracts* are reusable; only the *wiring* is compile-time. That is exactly the seam an extension-APK model must convert from compile-time to runtime.

2. The extension contract (IPC seam)

An external installable APK exposes a provider to Lava through an **exported ContentProvider** (stable IPC surface, survives process boundaries, queryable by `PackageManager`). The provider ships a `Parcelable` manifest + marshals feature calls as JSON-over-`Bundle`.

```
// core/tracker/extension-api/ (NEW shared lib – shipped in the
// app AND depended on by every extension APK)
```

```
@Parcelize
```

```
data class TrackerExtensionManifest(
    val trackerId: String,
    val displayName: String,
    val capabilities: List<String>, // TrackerCapability names;
    // string-typed to survive enum drift
    val authType: String, // AuthType name
    val version: Int, // extension's own version
    val minAppVersion: Int, // lowest Lava versionCode
    // this extension supports
    val sdkContract: Int, // the extension-api
    // contract version (handshake)
) : Parcelable
```

```
abstract class TrackerExtensionProvider : ContentProvider() {
    // Lava calls these over the ContentProvider boundary (call()
    // method, Bundle in/out, JSON payloads):
    abstract fun manifest(): TrackerExtensionManifest
    abstract fun search(trackerId: String, query: String, page:
        Int): ByteArray // JSON SearchResult
    abstract fun topic(trackerId: String, id: String):
        ByteArray // JSON TopicDetail
    abstract fun downloadTorrent(trackerId: String, id: String):
        ByteArray // raw .torrent bytes
    abstract fun magnet(trackerId: String, id: String):
        String? // magnet URI or null
    abstract fun login(trackerId: String, credsJson: ByteArray):
        ByteArray // JSON LoginResult
    // ContentProvider.call(method, arg, extras) dispatches to the
    // above by `method` name.
}
```

The intent/authority convention for discovery: authority prefix
digital.vasic.lava.tracker.extension.* + an <intent-filter> action
digital.vasic.lava.action.TRACKER_EXTENSION so Lava can enumerate
installed extensions.

3. Runtime discovery + registry merge

```
// core/tracker/extension-runtime/ (NEW, app-side)
```

```
class PluginDiscoveryService(private val pm: PackageManager,
    private val ctx: Context) {
    fun discover(): List<TrackerExtensionManifest> =
        pm.queryIntentContentProviders(Intent(ACTION_TRACKER_EXTENSION),
            0)
            .map { resolveManifestViaContentProvider(it) }
            .filter { it.minAppVersion <= BuildConfig.VERSION_CODE
                && it.sdkContract == SDK_CONTRACT }
}
```

```
// A ProxyTrackerClient wraps each discovered extension:
//     getFeature<T>() returns proxy feature
// impls that marshal the call over the ContentProvider and
//     deserialize the JSON/bytes back into
// the SAME core/tracker/api model types the in-process clients
//     return.
class ProxyTrackerClient(manifest, providerClient) :
    TrackerClient { ... }
```

TrackerClientModule.provideTrackerRegistry(...) is extended to **merge** discovered extensions into the registry alongside the existing compile-time factories — the in-process providers stay exactly as they are (zero regression risk); extensions are additive.

4. The refactor path (concrete files)

Step	Add / modify	Why
1	add core/tracker/ extension-api/ (Android lib)	Parcelable manifest + TrackerExtensionProvider + the JSON DTO contract + SDK_CONTRACT const
2	add core/tracker/ extension-runtime/	PluginDiscoveryService, ProxyTrackerClient, the marshalling codecs
3	modify core/tracker/ client/di/ TrackerClientModule.kt	merge PluginDiscoveryService.discover() results into DefaultTrackerRegistry after the 6 compile-time register() calls
4	modify settings.gradle.kts	include(":core:tracker:extension- api", ":core:tracker:extension- runtime")
5	add a reference extension APK module (out-of-tree or samples/)	proves the contract end-to-end with a real installable APK

5. Trade-offs (honest)

- **APK size:** each extension re-bundles Kotlin stdlib (~2 MB), OkHttp (~1.5 MB), Jsoup (~0.5 MB) → **≥5 MB per extension**. Unavoidable across the process boundary.
- **Dependency skew:** extension built against OkHttp 4.11 vs app 4.9 — both load in their own process; the IPC DTO contract (not shared classes) is what must stay stable.
- **Capability-enum drift:** capabilities is List<String> over IPC precisely so an extension built against an older enum doesn't crash; unknown capability strings are ignored by the host.
- **Version handshake:** sdkContract + minAppVersion gate which extensions load — a mismatch is skipped with a user-visible "extension needs a newer app" notice, never a crash.

- **Play-Store policy risk** (UNCONFIRMED – requires legal/policy review): a torrent-tracker *extension* model may face the same store-policy friction as the main app; community-published extensions on a different developer account spread that risk but also fragment trust. An out-of-store side-load / F-Droid-style channel is the lower-risk distribution for tracker extensions.

6. Prior art

UNCONFIRMED (from memory): **Tachiyomi/Mihon** (manga readers) use exactly this model — manga *source* extensions are separate installable APKs discovered by package query; each extension carries its own HTTP/parsing stack; the host marshals a small source-interface across the boundary. Lava's domain (tracker sources) maps cleanly onto it. This should be verified against the current Mihon extensions-lib before implementation.

7. Migration plan (phased) + native-vs-extension split

1. **Phase E1 (contract):** ship core/tracker/extension-api + a reference extension APK; prove one provider works as an installed extension end-to-end on the §6.AE device matrix.
2. **Phase E2 (discovery):** PluginDiscoveryService + ProxyTrackerClient + registry merge; existing 6 providers untouched.
3. **Phase E3 (migrate borderline providers):** providers with no Lava-specific coupling can move to extensions; **the core Russian trackers + the Jackett-backed path stay native/server-side** (they need the local-stack + FlareSolverr, which an on-device extension cannot host).

Split recommendation: native (in-app) for the curated core providers + the Jackett-sidecar breadth (server-side); **extension-APK** for community / long-tail providers where on-device, independently-published, independently-updated code is the value. IPTorrents specifically stays on the **Jackett+FlareSolverr** path (Cloudflare cannot be bypassed by an on-device OkHttp/Jsoup extension) — see docs/design/iptorrents-support.md.

Sources verified

- core/tracker/api/TrackerDescriptor.kt, TrackerClient.kt:16, TrackerCapability.kt, core/tracker/api/feature/*Tracker.kt, core/tracker/client/di/TrackerClientModule.kt:260–276 (the compile-time registry) — read during the 2026-06-08 decoupling audit.
- Tachiyomi/Mihon extension model — UNCONFIRMED (from memory); verify against current mihonapp/mihon source-api / extensions-lib before building.